

Université Paris-Est-Créteil
Laboratoire d'Algorithmique, Complexité et Logique

Mémoire de master

Un compilateur certifié de formules $\text{\LaTeX}$ en du code C++ utilisant l'API MPI

présenté par

Samir Gueblaoui

Master Logique Mathématiques et Fondement de l'Informatique (LMFI)
Université Paris Cité

Encadrement : Frédéric GAVA
Année universitaire : 2025–2026

Résumé

Ce mémoire étudie une chaîne qui transforme des formules mathématiques $\text{\LaTeX}$ en des programmes C++/MPI, puis réinjecte les résultats dans le document. L’objectif est de réduire la duplication entre texte scientifique et code, d’aider des utilisateurs non spécialistes du calcul parallèle. La contribution combine un prototype exécutable, un modèle de coût SEQ/PAR basé sur le modèle BSP, une génération C++/MPI et une formalisation Coq d’un fragment scalaire avec sommes finies.

À titre d’exemple, pour $n = 20$, notre compilateur évalue une troncature de la formule de Ramanujan et produit la valeur 3,1415926536 pour π , identique à la valeur attendue à la précision affichée.

```
\formulaToSkel[label=exemple]{\frac{1}{\frac{2\times\sqrt{2}}{9801}\times\sum_{i=0}^n\frac{(4i)!\times(1103+26390i)}{(i!)^4\times(396.0^4)^i}}}\$}
Attendu : $\approx \pi$. Obtenu : \soloValue{exemple}.
```

Entrée dans le document $\text{\LaTeX}$.

$$\frac{1}{\frac{2 \times \sqrt{2}}{9801} \times \sum_{i=0}^n \frac{(4i)! \times (1103 + 26390i)}{(i!)^4 \times (396.0^4)^i}}$$

Attendu : $\approx \pi$. Obtenu : 3.1415926536 (temps $p = 1$: 439495.042 μ s).

Résultat produit et réinséré.

Table des matières

Sigles et acronymes	iii
1 Introduction	1
1.1 Contexte	1
1.2 Question de recherche et périmètre	2
1.3 D’une formule vers du code	2
1.3.1 Exemple d’une formule naïve	2
1.3.2 BSP et parallélisation de l’exemple	3
1.3.3 Le modèle BSP	3
1.3.4 Parallélisation de l’exemple	3
1.3.5 De la spécification au C++/MPI	4
1.4 Chaîne proposée et plan	5
2 Sémantiques	7
2.1 Notations et conventions	7
2.2 Langage source	7
2.2.1 Syntaxe	8
2.2.2 Typage	8
2.2.3 Sémantique à grands pas	9
2.3 Propriétés de la sémantique source	10
2.4 Langage cible mini pseudo-MPI	10
2.4.1 Expressions, commandes et mémoires	10
2.4.2 Évaluation et transitions locales	11
2.4.3 Transition globale et réduction collective	12
2.4.4 Observation	13
3 Compilation	14
3.1 Choix entre génération de code séquentiel ou parallèle	14
3.2 Modèle de coût SEQ/PAR	14
3.2.1 Estimation récursive du travail	14
3.2.2 Coût séquentiel et coût parallèle simple	15
3.2.3 Exemple	15
3.3 Définition de la compilation	16
3.3.1 Principe général	16
3.3.2 Règles structurelles	16
3.3.3 Compilation d’une somme séquentielle	17
3.3.4 Compilation d’une somme parallèle simple	17
3.4 Énoncé de correction	18

3.5	Preuve de correction du cas directeur	18
4	Implantation et évaluation	21
4.1	Objectifs et méthode d'observation	21
4.2	Architecture de la chaîne	21
4.2.1	Composants logiciels	21
4.2.2	Deux parcours, un même cœur de compilation	22
4.3	Génération C++/MPI	23
4.3.1	Traduction par induction sur l'AST	23
4.3.2	Somme séquentielle	24
4.3.3	Somme parallèle simple	24
4.3.4	Sommes parallèles imbriquées	25
4.3.5	Assemblage du programme	27
5	Conclusion	29
5.1	Bilan	29
5.2	limitation et future travaux	29
	Références	31
A	Compléments techniques	32
A.1	Front-end ANTLR	32
A.1.1	Choix d'un analyseur généré	32
A.1.2	Insertion de la multiplication implicite	33
A.2	Intégration documentaire avec FORMEXEC	33
A.2.1	La formule comme source et comme élément du document	33
A.2.2	Cycle de construction	34
A.2.3	Complétion, compilation et protocole de sortie	34
A.3	Support initial des vecteurs	35
A.3.1	Représentation et accès	35
A.3.2	Chargement texte et Matrix Market	35
A.3.3	Périmètre exact	36
A.4	Exemples implémentés	36

Sigles et acronymes

ANTLR	<i>ANother Tool for Language Recognition</i> , générateur d'analyseurs.
API	<i>Application Programming Interface</i> , interface de programmation d'application.
AST	<i>Abstract Syntax Tree</i> , arbre de syntaxe abstraite.
BSP	<i>Bulk Synchronous Parallel</i> , modèle de calcul parallèle synchrone.
HPC	<i>High-Performance Computing</i> , calcul haute performance.
IEEE 754	<i>Institute of Electrical and Electronics Engineers</i> ; IEEE 754 est sa norme d'arithmétique flottante.
JSON	<i>JavaScript Object Notation</i> , format textuel de données.
MPI	<i>Message Passing Interface</i> , interface standard de communication par messages.
SIMD	<i>Single Instruction, Multiple Data</i> , exécution d'une instruction sur plusieurs données.
SPMD	<i>Single Program, Multiple Data</i> , modèle où les unités exécutent un même programme sur des données distinctes.

Chapitre 1

Introduction

1.1 Contexte

Dans de nombreux domaines expérimentaux, la production de connaissances passe désormais par un programme. Une mesure est filtrée, une population est simulée, un système différentiel est discrétisé ou une statistique est estimée par une suite de traitements numériques. Le résultat publié dépend alors à la fois du modèle mathématique et de sa traduction logicielle. Or ces deux objets sont souvent écrits dans des environnements séparés : l'article généralement écrit en LaTeX expose une équation en $\text{\LaTeX}$, tandis qu'un dépôt contient une seconde version programmée de la même équation. La cohérence entre les deux est entretenue manuellement.

Lorsque les simulations deviennent conséquentes ou doivent être répétées, le temps d'exécution et la mémoire d'une seule machine deviennent limitants. Le calcul distribué permet alors de mobiliser davantage d'unités de calcul et de mémoire. Il introduit cependant des difficultés absentes du code séquentiel : répartition des données et du travail, communications, synchronisations et reconstruction du résultat. Plusieurs bibliothèques fournissent de telles primitives ; ce mémoire retient l'interface standard de passage de messages MPI. Transformer une somme en programme distribué exige ainsi de répartir l'intervalle sans perdre ni dupliquer d'indice, d'appeler les communications collectives dans un ordre compatible et de choisir des types cohérents. Une traduction manuelle peut introduire des erreurs de borne, de communication ou de synchronisation, auxquelles s'ajoute la question de l'efficacité du parallélisme. Ces décisions sont familières à un spécialiste du calcul haute performance, moins à un biologiste, un physicien ou un statisticien dont l'objet premier demeure le modèle.

Le présent travail explore une autre approche, comparable à la *programmation lettrée* de Knuth [3] : la formule $\text{\LaTeX}$ devient le programme source. La *programmation lettrée* organise le code comme une explication destinée d'abord à l'humain ; le document exécutable rapproche texte, calcul et résultat afin de faciliter la reproductibilité ; enfin, un langage dédié restreint l'espace des programmes pour offrir une notation et des analyses adaptées à un domaine. L'objectif n'est pas de faire de $\text{\LaTeX}$ un langage généraliste, ni de remplacer le spécialiste du calcul haute performance HPC. Il est de donner un sens calculatoire à un fragment mathématique assez petit pour être analysé, compilé et raisonné formellement.

Le terme *sans code* est parfois employé pour des interfaces qui masquent le langage d'implantation. Ici, il doit être compris avec prudence. L'utilisateur écrit bien un programme, mais dans une notation mathématique déclarative : bornes, corps de réduction et paramètres. Les vrais codes C++/MPI, la distribution des calculs, leurs communications et synchronisation seront elles synthétisées depuis la formule. Cette approche partage avec le *low-code* la volonté de relever

le niveau d'abstraction, tout en donnant à la source une sémantique explicite et un chemin vers la preuve.

1.2 Question de recherche et périmètre

La problématique qui guide ce mémoire est la suivante :

Comment fournir à un scientifique un outil qui, à partir de formules mathématiques écrites en L^AT_EX, simplifie l'écriture, détecte certaines erreurs, produit un code C++/MPI séquentiel ou parallèle et réinjecte des résultats reproductibles dans le document, tout en donnant une justification formelle de la traduction ?

Cette question associe quatre exigences qui ne se confondent pas.

1. La formule doit conserver une apparence familière et éviter à l'auteur d'écrire les mécanismes MPI.
2. Un analyseur doit construire un arbre, vérifier les contraintes de types et refuser les constructions que le générateur ne sait pas traiter.
3. Le choix séquentiel ou parallèle ne peut reposer sur l'intuition seule ; il doit résulter d'un modèle de coût paramétrable, même simple.
4. La formalisation doit dire précisément quel fragment est couvert, quelles hypothèses relient source et cible et quelles propriétés restent ouvertes.

1.3 D'une formule vers du code

1.3.1 Exemple d'une formule naïve

Considérons la formule :

$$S(n) = \sum_{i=0}^n f(i). \quad (1.1)$$

Elle paraît élémentaire, mais concentre les principaux choix du projet. Il faut interpréter les bornes comme inclusives, évaluer f dans un environnement où i est lié à la valeur courante, définir le résultat lorsque $n < 0$, choisir un type pour l'accumulateur, puis décider si l'intervalle est suffisamment grand pour justifier une exécution distribuée.

En notant :

$$m = \max(n - 0 + 1, 0) \quad (1.2)$$

le nombre d'itérations, l'algorithme séquentiel est une réduction à gauche dont l'accumulateur est initialisé à zéro. Sa forme abstraite est donnée par le Listing 1.1.

```
1 T sum(Term f, int n) {
2     T acc = 0;
3     for (int i = 0; i <= n; ++i) {
4         acc = acc + f(i);
5     }
6     return acc;
7 }
```

Listing 1.1 : Pseudo-code séquentiel de la somme finie.

Le type générique T peut représenter ici un entier ou un flottant. Cette boucle est déjà une décision de compilation : elle fixe l'ordre des additions. Sur les entiers mathématiques,

l'associativité permet de réorganiser la somme. Sur des nombres flottants IEEE 754, cet argument algébrique n'implique pas l'égalité bit à bit, car l'arrondi rend l'addition non associative [2]. Une exécution parallèle pourra donc être correcte au niveau du modèle réel de Coq et néanmoins produire un dernier bit différent en C++. Les méthodes de sommation reproductible traitent spécifiquement ce problème [1] ; elles ne sont pas implantées ici.

1.3.2 BSP et parallélisation de l'exemple

1.3.3 Le modèle BSP

Le modèle *Bulk Synchronous Parallel* (BSP), introduit par Valiant [7], fournit une représentation abstraite d'une machine parallèle. Celle-ci est composée de $\mathbf{P}$ unités de calcul, chacune disposant de sa propre mémoire locale, et d'un réseau permettant l'échange de données. BSP ne décrit pas précisément le matériel ni l'implantation des communications : il fournit plutôt un cadre commun pour raisonner sur l'organisation et le coût d'un algorithme parallèle.

Une exécution BSP est divisée en *super-étapes*. Chaque super-étape comporte trois phases :

1. chaque unité réalise des calculs sur ses données locales ;
2. les unités échangent les données nécessaires à la suite du calcul ;
3. une barrière de synchronisation garantit que toutes les communications de la super-étape sont terminées avant de poursuivre.

Les calculs locaux peuvent être exécutés indépendamment et dans un ordre quelconque. En revanche, la barrière impose que l'unité la plus rapide attende la plus lente avant le début de la super-étape suivante. Le coût du calcul local est donc déterminé par l'unité qui effectue le plus de travail, et non par la moyenne des travaux.

Le coût d'une super-étape peut être représenté par :

$$T_{\text{BSP}} = w + \mathbf{g}h + \mathbf{L}, \quad (1.3)$$

où w désigne le temps maximal de calcul local, h le volume maximal de données envoyé ou reçu par une unité, $\mathbf{g}$ le coût de communication d'une unité de donnée et $\mathbf{L}$ le coût de la synchronisation.

Les paramètres $\mathbf{g}$ et $\mathbf{L}$ représentent les caractéristiques de la machine cible. Ils peuvent être estimés expérimentalement par des micro-benchmarks.

Dans ce mémoire, BSP sert principalement de modèle de coût pour comparer une exécution séquentielle et une exécution distribuée.

1.3.4 Parallélisation de l'exemple

La somme de l'équation (1.1) se décompose naturellement en deux squelettes algorithmiques :

$$[0, n] \xrightarrow{\text{map}(f)} [f(0), \dots, f(n)] \xrightarrow{\text{reduce}(+, 0)} S(n). \quad (1.4)$$

Les squelettes capturent des schémas parallèles récurrents et permettent de séparer la structure d'exécution du calcul propre au domaine.

La distribution choisie est cyclique (on aurait pu aussi faire une distribution par bloc). Le rang $r \in \{0, \dots, \mathbf{P} - 1\}$ traite :

$$I_r = \{i \in [0, n] \mid i \equiv r \pmod{\mathbf{P}}\}. \quad (1.5)$$

Les ensembles I_r sont deux à deux disjoints et leur union est l'intervalle complet. Chaque rang calcule une somme locale ; une réduction globale additionne les $\mathbf{P}$ résultats. La structure correspond à une super-étape BSP : travail local indépendant, communication collective, puis disponibilité du résultat sur tous les rangs.

Notons $\mathcal{C}(e)$ le coût estimé de l'expression e . Soit $\mathcal{C}(f(i))$ le coût du corps, c_+ celui d'une accumulation, $\mathbf{P}$ le nombre de processus supposé, $\mathbf{g}$ le coût d'une unité de communication, V le volume réduit et $\mathbf{L}$ une latence de synchronisation. Le prototype compare les estimations :

$$C_{\text{seq}} = \mathcal{C}(0) + \mathcal{C}(n) + m(\mathcal{C}(f(i)) + c_+), \quad (1.6)$$

$$C_{\text{par}} = C(0) + C(n) + \left\lceil \frac{m}{\mathbf{P}} \right\rceil (\mathcal{C}(f(i)) + c_+) + \mathbf{P}\mathbf{g}V + \mathbf{L}. \quad (1.7)$$

Ce modèle est volontairement élémentaire. Il constitue une fonction de décision inspectable, pas une prédiction universelle. Une somme n'est intéressante à parallélisée que si $C_{\text{par}} < C_{\text{seq}}$ et si son intervalle dépasse un seuil minimal.

1.3.5 De la spécification au C++/MPI

MPI (*Message Passing Interface*) est une interface standard destinée à l'exécution de programmes sur plusieurs processus possédant chacun leur propre mémoire [4]. Nous nous plaçons dans le modèle SPMD (*Single Program, Multiple Data*) : tous les processus exécutent le même programme, mais traitent des données différentes selon leur rang (aussi appelé pid=identifiant pour "processus identifiant"). Chaque processus est identifié par ce rang dans un communicateur et effectue une partie du calcul sur ses données locales. Les processus échangent ensuite leurs résultats au moyen de communications point à point ou d'opérations collectives telles que ALLREDUCE, qui combine les valeurs locales et transmet le résultat obtenu à tous les participants.

Le Listing 1.2 donne un programme complet pour $\sum_{i=0}^n f(i)$. L'exemple choisit $f(i) = \sin(i)$ et $n = 20$.

```

1  #include <mpi.h>
2  #include <cmath>
3  #include <iostream>
4  #include <iomanip>
5
6  double f(int i) {
7      return std::sin(static_cast<double>(i));
8  }
9
10 int main(int argc, char** argv) {
11     MPI_Init(&argc, &argv);
12
13     int rank = 0;
14     int size = 1;
15     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
16     MPI_Comm_size(MPI_COMM_WORLD, &size);
17
18     const int n = 20;
19     double local = 0.0;
20     for (int i = rank; i <= n; i += size) {
21         local += f(i);
22     }
23
24     double global = 0.0;

```

```

25 MPI_Allreduce(&local, &global, 1,
26               MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
27
28 if (rank == 0) {
29     std::cout << std::setprecision(10)
30               << global << std::endl;
31 }
32
33 MPI_Finalize();
34 return 0;
35 }

```

Listing 1.2 : Programme C++/MPI complet d'une somme parallèle cyclique.

ALLREDUCE applique ici `MPI_SUM` et rend le résultat à chaque membre du communicateur, conformément à la spécification MPI.

Plusieurs obligations se cachent derrière ce programme.

- Tous les rangs du communicateur doivent atteindre la collective dans un ordre compatible.
- Le type MPI doit correspondre au type de l'accumulateur.
- La partition cyclique ne doit ni perdre ni dupliquer un indice.
- L'expression $f(i)$ ne doit pas modifier une donnée partagée d'une façon non représentée dans le modèle source.
- La réduction parallèle doit être reliée à la sémantique de la somme.

1.4 Chaîne proposée et plan

Le projet associe trois artefacts reliés par une même formule, comme l'illustre la Figure 1.1.

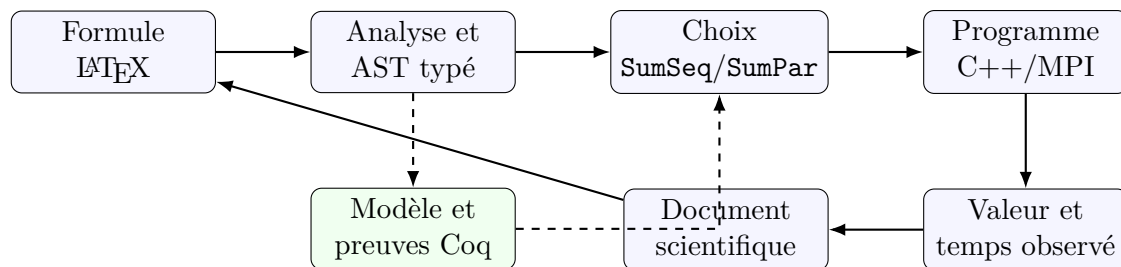


Figure 1.1 : Vue d'ensemble : compilation exécutable et formalisation partagent les concepts, sans preuve actuelle de correspondance entre les artefacts C++ et Coq (flèches pointillées).

Les contributions étudiées dans ce mémoire sont les suivantes.

1. Un fragment de formules `LaTeX` est analysé par ANTLR, converti dans un arbre interne et soumis à une vérification de types.
2. Les sommes reçoivent une stratégie SUMSEQ ou SUMPAR à partir d'un modèle de coût inspiré de BSP, configurable et réévalué après spécialisation des paramètres.
3. Un générateur produit du C++/MPI, avec distribution cyclique et réduction collective pour le cas parallèle.
4. Les macros FORMEXEC raccordent la formule, la configuration JSON, la compilation, l'exécution et l'insertion de la valeur dans le PDF.
5. Un développement Coq formalise le langage source et un pseudo-MPI, prouve le déterminisme conditionnel de la sémantique source, une simulation locale, la correction du fragment sans somme parallèle et d'une somme parallèle simple.

Le [chapitre 2](#) définit la sémantique des formules et celle du pseudo-code distribué, puis examine les théorèmes de déterminisme et l'absence actuelle d'un théorème d'existence. Le [chapitre 3](#) décrit le modèle de coût, l'algorithme SUMSEQ/SUMPAR et la preuve de correction. Le [chapitre 4](#) relie ces objets aux grammaires ANTLR, aux deux AST, au générateur C++/MPI, aux macros et aux exemples, avec une attention particulière aux vecteurs et aux limites observées. Enfin, le [chapitre 5](#) synthétise les garanties acquises et propose un programme de travaux futurs : données distribuées, modèles de coût calibrés, matrices et tenseurs, cibles GPU/SIMD et certification de bout en bout.

Chapitre 2

Sémantiques

Ce chapitre définit le sens d'un mini langage de formules $\text{L}\text{A}\text{T}\text{E}\text{X}$ et un mini langage du pseudo-code avant d'étudier leur traduction. Une sémantique à grands pas décrit les formules. Une sémantique à petits pas décrit ensuite le pseudo-code. Cette séparation permet de distinguer la valeur mathématique attendue de la manière dont le programme la calcule. La formalisation est vérifiée avec Coq, assistant de preuve interactif dont le noyau contrôle les termes de preuve [6].

2.1 Notations et conventions

Le modèle porte sur des expressions scalaires entières ou réelles. Il contient des constantes, des variables, des fonctions, des opérations arithmétiques et des sommes finies. Les booléens appartiennent seulement au langage cible. Ils servent aux tests de boucle et aux conditions. Les tableaux, matrices et tenseurs admis partiellement par le prototype ne sont pas inclus dans cette sémantique.

Les notations suivantes seront conservées dans les chapitres sémantique et compilation :

$\mathbb{N}, \mathbb{Z}, \mathbb{R}$	naturels, entiers et réels mathématiques,
$\text{Int}, \text{Float}, \text{Bool}$	types du modèle,
Γ	environnement de typage source,
Γ_c	environnement de typage cible,
ρ	environnement de valeurs source,
σ	mémoire locale d'un processus,
Φ, Ψ	signatures unaires et binaires ($\mathbb{U}, \mathbb{B}$ dans Coq),
$\Sigma = (\sigma_0, \dots, \sigma_{p-1})$	état mémoire global,
κ, ε	continuation locale et continuation vide.

Une fonction partielle est notée $A \rightarrow B$. La mise à jour fonctionnelle s'écrit $\rho[x \mapsto v]$ ou $\sigma[x \mapsto v]$. L'expression R^* désigne la clôture réflexive et transitive de la relation R . Enfin, $\oplus$ et $\otimes$ représentent l'addition et la multiplication avec promotion numérique.

2.2 Langage source

Le fragment couvert par le théorème de correction comprend les expressions sans somme parallèle imbriqué et les expressions dont l'unique somme parallèle est à la racine. Dans ce second cas, les bornes et le corps ne contiennent pas de nouvelle somme parallèle. Le compilateur définit aussi

un traitement des sommes parallèles imbriquées, mais la preuve globale ne couvre pas ce cas. Le mini-langage source et sa sémantique sont formalisés dans le fichier Coq `C00_Source.v`.

2.2.1 Syntaxe

Le langage source possède deux types :

$$\tau ::= \text{Int} \mid \text{Float}.$$

Ses expressions sont définies par la grammaire suivante :

$$e ::= c_{\mathbb{Z}} \mid c_{\mathbb{R}} \mid x \mid f(e) \mid e + e \mid e \times e \mid y(e, e) \\ \mid \sum_{i=a}^b e \mid \sum_{i=a}^b e.$$

$\text{seq} \qquad \text{par}$

Ici, f désigne une fonction unaire, y une fonction binaire et i la variable liée par la somme. Les annotations `seq` et `par` ne changent pas le sens mathématique de la somme. Elles indiquent la stratégie de compilation souhaitée.

2.2.2 Typage

L'environnement de typage et les signatures fonctionnelles sont notés :

$$\Gamma : \text{Var} \rightarrow \{\text{Int}, \text{Float}\}, \quad \Phi : \text{Fonc} \rightarrow \{\text{Int}, \text{Float}\} \rightarrow \text{option}(\{\text{Int}, \text{Float}\}),$$

$$\Psi : \text{FoncBin} \rightarrow \{\text{Int}, \text{Float}\}^2 \rightarrow \text{option}(\{\text{Int}, \text{Float}\}).$$

La promotion P vérifie $P(\text{Int}, \text{Int}) = \text{Int}$. Tous les autres couples numériques produisent `Float`. Le jugement $\Gamma; \Phi; \Psi \vdash e : \tau$ signifie que e possède le type τ .

Les règles des constantes, des variables et de l'addition sont :

$$\frac{c \in \mathbb{Z}}{\Gamma; \Phi; \Psi \vdash c : \text{Int}} \quad \frac{r \in \mathbb{R}}{\Gamma; \Phi; \Psi \vdash r : \text{Float}} \quad \frac{\Gamma(x) = \text{Some}(\tau)}{\Gamma; \Phi; \Psi \vdash x : \tau},$$

$$\frac{\Gamma; \Phi; \Psi \vdash e_1 : \tau_1 \quad \Gamma; \Phi; \Psi \vdash e_2 : \tau_2 \quad \tau = P(\tau_1, \tau_2)}{\Gamma; \Phi; \Psi \vdash e_1 + e_2 : \tau}.$$

La multiplication suit la même règle. Pour une fonction unaire, on exige $\Phi(f, \tau) = \text{Some}(\tau')$. Une primitive binaire exige de même que $\Psi(y, \tau_1, \tau_2)$ retourne un type.

Les deux formes de somme partagent enfin la règle :

$$\frac{\Gamma; \Phi; \Psi \vdash a : \text{Int} \quad \Gamma; \Phi; \Psi \vdash b : \text{Int} \quad \Gamma[i \mapsto \text{Int}]; \Phi; \Psi \vdash e : \tau}{\Gamma; \Phi; \Psi \vdash \sum_{i=a}^b e : \tau} \quad (* \in \{\text{seq}, \text{par}\}).$$

La différence entre les deux constructeurs relève donc de la compilation, et non du type ou de la valeur de la formule. La syntaxe des expressions et la relation de typage `has_type` sont définies dans le fichier Coq `C00_Source.v`.

2.2.3 Sémantique à grands pas

Les valeurs source du code Coq portent leur constructeur :

$$v ::= \text{VInt}(n) \mid \text{VFloat}(r), \quad n \in \mathbb{Z}, \quad r \in \mathbb{R}.$$

L'environnement dynamique est $\rho : \text{Var} \rightarrow \text{option}(\text{value})$. Les interprétations source sont des fonctions totales abstraites :

$$\overline{\mathcal{U}}_s : \text{Fonc} \rightarrow \text{value} \rightarrow \text{value}, \quad \overline{\mathcal{B}}_s : \text{FoncBin} \rightarrow \text{value}^2 \rightarrow \text{value}.$$

Elles sont distinctes des signatures Φ, Ψ . Les règles d'évaluation vérifient que leurs résultats possèdent le type annoncé.

Pour :

$$\Gamma; \Phi; \Psi; \rho \vdash e \Downarrow_{\tau} v$$

nous omettrons les paramètres fixes et écrirons simplement $\rho \vdash e \Downarrow v$. Les règles élémentaires sont :

$$\frac{}{\Gamma; \Phi; \Psi; \rho \vdash c \Downarrow_{\text{Int}} \text{VInt}(c)} \quad \frac{\Gamma(x) = \text{Some}(\tau) \quad \rho(x) = \text{Some}(v) \quad v : \tau}{\Gamma; \Phi; \Psi; \rho \vdash x \Downarrow_{\tau} v} \quad \frac{\rho \vdash e \Downarrow v \quad \Phi(f, \tau) = \text{Some}(\tau') \quad \mathcal{U}_s(f, v) : \tau'}{\rho \vdash f(e) \Downarrow \overline{\mathcal{U}}_s(f, v)}.$$

Pour les opérations arithmétiques :

$$\frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho \vdash e_2 \Downarrow v_2}{\rho \vdash e_1 + e_2 \Downarrow v_1 \oplus v_2}, \quad \frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho \vdash e_2 \Downarrow v_2}{\rho \vdash e_1 \times e_2 \Downarrow v_1 \otimes v_2}.$$

Pour définir la sémantique des sommes, nous introduisons le jugement auxiliaire :

$$\Gamma; \Phi; \Psi; \rho \vdash \text{Sum}(i, m, n, e) \Downarrow_{\tau} v.$$

Il signifie que l'évaluation du corps e , lorsque i parcourt l'intervalle entier $[m, n]$, produit la valeur v de type τ .

Lorsque l'intervalle est vide, la somme vaut l'élément neutre 0_{τ} :

$$\frac{n < m}{\Gamma; \Phi; \Psi; \rho \vdash \text{Sum}(i, m, n, e) \Downarrow_{\tau} 0_{\tau}}.$$

Lorsque $m \leq n$, le corps est évalué dans l'environnement où i reçoit la valeur entière m , puis cette valeur est ajoutée au résultat de la somme restante :

$$\frac{m \leq n \quad \Gamma; \Phi; \Psi; \rho[i \mapsto \text{VInt}(m)] \vdash e \Downarrow_{\tau} v_m \quad \Gamma; \Phi; \Psi; \rho \vdash \text{Sum}(i, m+1, n, e) \Downarrow_{\tau} v_{\text{reste}}}{\Gamma; \Phi; \Psi; \rho \vdash \text{Sum}(i, m, n, e) \Downarrow_{\tau} (v_m \oplus v_{\text{reste}})}.$$

La somme séquentielle est alors évaluée par la règle suivante :

$$\frac{\rho \vdash a \Downarrow_{\text{Int}} \text{VInt}(m) \quad \rho \vdash b \Downarrow_{\text{Int}} \text{VInt}(n) \quad \Gamma; \Phi; \Psi; \rho \vdash \text{Sum}(i, m, n, e) \Downarrow_{\tau} v}{\rho \vdash \sum_{\text{seq}}^b_{i=a} e \Downarrow_{\tau} v}.$$

La somme parallèle possède la même règle sémantique :

$$\frac{\begin{array}{c} \rho \vdash a \Downarrow_{\text{Int}} \text{VInt}(m) \quad \rho \vdash b \Downarrow_{\text{Int}} \text{VInt}(n) \\ \Gamma; \Phi; \Psi; \rho \vdash \text{Sum}(i, m, n, e) \Downarrow_{\tau} v \end{array}}{\rho \vdash \sum_{\text{par}}^{b}_{i=a} e \Downarrow_{\tau} v}.$$

La sémantique à grands pas des expressions et des sommes correspond aux relations `Coq eval_expr` et `eval_sum`, définies dans `C00_Source.v`.

2.3 Propriétés de la sémantique source

Théorème 2.1 (Type d’une évaluation).

$$e : \tau \quad \wedge \quad e \Downarrow v \quad \Longrightarrow \quad v : \tau.$$

Théorème 2.2 (Déterminisme des formules).

$$\Gamma; \Phi; \Psi; \rho \vdash e \Downarrow_{\tau_1} v_1 \quad \wedge \quad \Gamma; \Phi; \Psi; \rho \vdash e \Downarrow_{\tau_2} v_2 \quad \Longrightarrow \quad v_1 = v_2.$$

La préservation du type des valeurs est démontrée dans `C03_Invariants.v`, tandis que le déterminisme de la sémantique source est démontré dans `C09_FormulaDeterminism.v`.

2.4 Langage cible mini pseudo-MPI

2.4.1 Expressions, commandes et mémoires

Le langage cible ajoute des valeurs booléennes :

$$cv ::= \text{CVInt}(n) \mid \text{CVFloat}(r) \mid \text{CVBool}(b).$$

Pour la formule directrice, le fragment utile des expressions est :

$$\begin{aligned} a ::= & n \mid r \mid b \mid x \mid f^{\sharp}(a) \mid a + a \mid a \times a \mid a - a \mid a \bmod a \\ & \mid a \leq a \mid a == a \mid \neg a \mid a \wedge a. \end{aligned}$$

On pourrais avoir $f^{\sharp}(a_1, \dots, a_n)$ mais par simplicité d’écriture on se limite à des fonctions unaire (mais rien ne limite à des fonctions avec plus d’arguments). Les principales règles s’écrivent :

$$\begin{array}{c} \frac{n \in \mathbb{Z}}{\Gamma_c \vdash n : \text{Int}} \quad \frac{r \in \mathbb{R}}{\Gamma_c \vdash r : \text{Float}} \quad \frac{\Gamma_c(x) = \tau_c}{\Gamma_c \vdash x : \tau_c}, \\ \frac{\Gamma_c \vdash a_1 : \tau_1 \quad \Gamma_c \vdash a_2 : \tau_2 \quad \tau = P(\tau_1, \tau_2)}{\Gamma_c \vdash a_1 + a_2 : \tau}. \end{array}$$

Les opérations classiques ce type comme l’addition. Une comparaison exige deux types numériques et produit `Bool`. Enfin,

$$\frac{\Gamma_c \vdash a : \text{Bool}}{\Gamma_c \vdash \neg a : \text{Bool}} \quad \frac{\Gamma_c \vdash a_1 : \text{Bool} \quad \Gamma_c \vdash a_2 : \text{Bool}}{\Gamma_c \vdash a_1 \wedge a_2 : \text{Bool}}.$$

Le pseudo-code est défini par :

$$c ::= \mathbf{skip} \mid x := a \mid c; c \mid \mathbf{if} \ a \ \mathbf{then} \ c \ \mathbf{else} \ c \\ \mid \mathbf{for}(c_{\text{init}}; a_{\text{test}}; c_{\text{step}})\{c\} \mid \mathbf{ALLREDUCE}(x, y).$$

Une mémoire locale est :

$$\sigma : \text{VarCible} \rightarrow \text{option}(\text{cvalue}).$$

Pour $p \geq 1$ unité de calculs (processeurs), l'état global est $\Sigma = (\sigma_0, \dots, \sigma_{p-1})$. Chaque mémoire contient *rank*, *size* et *active*, avec $\sigma_r(\text{rank}) = \text{Some}(\text{CVInt}(r))$ et $\sigma_r(\text{size}) = \text{Some}(\text{CVInt}(p))$.

Une continuation décrit le travail restant :

$$\kappa ::= \varepsilon \mid c \cdot \kappa \mid \text{del}[x_1, \dots, x_m] \cdot \kappa.$$

Un état local est $\langle \sigma, \kappa \rangle$. Une configuration globale réunit un état local par processus. L'action *del* retire les variables temporaires créées dans une branche ou une itération. Le langage cible abstrait utilisé pour représenter le code pseudo-MPI est défini dans le fichier Coq `C01_TargetCompiler.v`.

2.4.2 Évaluation et transitions locales

La valeur partiellement évalué d'une expression cible est notée $\llbracket a \rrbracket_\sigma$. Par exemple,

$$\llbracket x \rrbracket_\sigma = \sigma(x), \quad \llbracket a_1 + a_2 \rrbracket_\sigma = \overline{\llbracket a_1 \rrbracket_\sigma + \llbracket a_2 \rrbracket_\sigma}.$$

Elle retourne **None** pour une variable absente, un modulo par zéro ou une opération appliquée à des valeurs incompatibles.

La transition locale s'écrit :

$$\langle \sigma, \kappa \rangle \rightarrow_\ell \langle \sigma', \kappa' \rangle.$$

Les règles fondamentales sont :

$$\frac{\llbracket a \rrbracket_\sigma = \text{Some}(cv)}{\langle \sigma, (x := a) \cdot \kappa \rangle \rightarrow_\ell \langle \sigma[x \mapsto cv], \mathbf{skip} \cdot \kappa \rangle}, \\ \langle \sigma, (c_1; c_2) \cdot \kappa \rangle \rightarrow_\ell \langle \sigma, c_1 \cdot c_2 \cdot \kappa \rangle, \quad \langle \sigma, \mathbf{skip} \cdot \kappa \rangle \rightarrow_\ell \langle \sigma, \kappa \rangle, \\ \langle \sigma, \text{del}[X] \cdot \kappa \rangle \rightarrow_\ell \langle \sigma \setminus X, \kappa \rangle.$$

Une condition choisit la branche correspondant à la valeur booléenne du test, puis place son nettoyage dans la continuation. Une boucle exécute l'initialisation une fois. Tant que le test est vrai, elle empile le corps, le nettoyage, le pas d'incrément et la boucle restante. Lorsque le test est faux, elle produit **skip**. Une continuation vide est terminale.

Les règles correspondante sont :

$$\frac{\llbracket a \rrbracket_\sigma = \text{Some}(\text{CVBool}(\mathbf{true}))}{\langle \sigma, (\mathbf{if} \ a \ \mathbf{then} \ c_t^{[X]} \ \mathbf{else} \ c_f^{[Y]}) \cdot \kappa \rangle \rightarrow_\ell \langle \sigma, c_t \cdot \text{del}[X] \cdot \kappa \rangle} \\ \frac{\llbracket a \rrbracket_\sigma = \text{Some}(\text{CVBool}(\mathbf{false}))}{\langle \sigma, (\mathbf{if} \ a \ \mathbf{then} \ c_t^{[X]} \ \mathbf{else} \ c_f^{[Y]}) \cdot \kappa \rangle \rightarrow_\ell \langle \sigma, c_f \cdot \text{del}[Y] \cdot \kappa \rangle}.$$

Pour une boucle déjà initialisée, un test vrai produit :

$$\begin{aligned} & \langle \sigma, \mathbf{for}(\mathbf{skip}; a_{\text{test}}; c_{\text{step}}) \{c\}^{[X]} \cdot \kappa \rangle \\ & \rightarrow_{\ell} \langle \sigma, c \cdot \text{del}[X] \cdot c_{\text{step}} \cdot \mathbf{for}(\mathbf{skip}; a_{\text{test}}; c_{\text{step}}) \{c\}^{[X]} \cdot \kappa \rangle, \end{aligned}$$

sous la prémisse $\llbracket a_{\text{test}} \rrbracket_{\sigma} = \text{Some}(\text{CVBool}(\mathbf{true}))$. Si le test retourne $\text{CVBool}(\mathbf{false})$, la boucle est remplacée par **skip**. Ces règles rendent explicite l'ordre corps-nettoyage-incrément.

Théorème 2.3 (Déterminisme local).

$$s \rightarrow_{\ell} s_1 \quad \wedge \quad s \rightarrow_{\ell} s_2 \quad \implies \quad s_1 = s_2.$$

où $\rightarrow_{\ell}$ est une équivalence structurelle.

La partie évaluation, transitions locale et le déterminisme locale est dans le fichier `C02_SmallStep.v`.

2.4.3 Transition globale et réduction collective

Un pas local d'un processus induit un pas global, noté $\implies$, qui laisse les autres états inchangés. Plusieurs processus peuvent être prêts en même temps ; cette relation globale autorise donc plusieurs entrelacements.

La commande $\text{ALLREDUCE}(x, y)$ n'a pas de pas local. Elle s'exécute lorsque tous les processus attendent le même collectif. Si

$$V = \sum_{r=0}^{p-1} \sigma_r(x),$$

la transition écrit V dans y sur chaque rang et retire la commande de chaque continuation :

$$\begin{aligned} & \langle \sigma_0, \text{ALLREDUCE}(x, y) \cdot \kappa'_0; \dots; \sigma_{p-1}, \text{ALLREDUCE}(x, y) \cdot \kappa'_{p-1} \rangle \\ & \implies \langle \sigma_0[y \mapsto V], \kappa'_0; \dots; \sigma_{p-1}[y \mapsto V], \kappa'_{p-1} \rangle. \end{aligned}$$

Nous notons $G \rightarrow_{\text{BSP}} G'$ l'exécution globale déterministe retenue. Alors que $\implies$ autorise plusieurs entrelacements des pas locaux, $\rightarrow_{\text{BSP}}$ en fixe un ordre unique pour obtenir une même exécution à partir d'une configuration donnée.

Pour $\Sigma = (\sigma_0, \dots, \sigma_{p-1})$, notons :

$$\text{finalStates}(\Sigma) = \langle \sigma_0, \varepsilon; \dots; \sigma_{p-1}, \varepsilon \rangle.$$

Une terminaison *réussie* atteint une configuration de cette forme.

Théorème 2.4 (Déterminisme de l'exécution BSP). *Soient G une configuration globale initiale et G_1, G_2 deux configurations globales terminales alors*

$$G \rightarrow_{\text{BSP}}^* G_1 \quad \wedge \quad G \rightarrow_{\text{BSP}}^* G_2 \quad \implies \quad G_1 = G_2.$$

La fonction d'exécution BSP, son lien avec les transitions globales et ses résultats de déterminisme sont formalisés dans `C10_ProgramDeterminism.v`.

La correction requiert que les interprétations source et cible des primitives coïncident. Le prédicat `Coq PrimitiveCompatible`(Φ, Ψ) exige :

$$\overline{\mathcal{U}}_t(f, v) = \overline{\mathcal{U}}_s(f, v), \quad \overline{\mathcal{B}}_t(y, v_1, v_2) = \overline{\mathcal{B}}_s(y, v_1, v_2).$$

Le modèle utilise $\mathbb{Z}$ et $\mathbb{R}$. Il ignore donc les débordements des entiers machine et les arrondis IEEE-754. En particulier, l'addition flottante n'est pas associative ; une réduction MPI peut changer de résultat lorsque l'arbre de réduction ou le nombre de processus change.

2.4.4 Observation

Le jugement $\Gamma; \Phi; \Psi; \rho \vdash e \Downarrow_{\tau} v$ fixe la valeur source ; $\rightarrow_{\ell}$ décrit un processus et $\Longrightarrow$ leur exécution collective, notamment ALLREDUCE. Les deux sommes ont la même sémantique, mais des compilations distinctes. Coq prouve l'unicité d'une évaluation existante et d'un état final BSP atteint, sans certifier le parseur, le C++/MPI, les sommes parallèles imbriquées ni l'arithmétique machine. Ce périmètre fonde l'énoncé de correction du chapitre suivant.

Chapitre 3

Compilation

3.1 Choix entre génération de code séquentiel ou parallèle

Compiler une somme requiert ici deux décisions de nature différente. La première est une décision d'optimisation : conserver une boucle séquentielle ou distribuer les itérations. Elle dépend d'une estimation de coût et peut changer avec les paramètres connus. La seconde est une décision sémantique : produire un code dont le résultat représente la valeur de la formule. Elle doit rester vraie quelle que soit la stratégie retenue dans le fragment couvert.

Cette séparation apparaît directement dans l'implantation. Lors de la conversion de l'arbre `EXPRLATEX`, une somme est d'abord construite comme une `SUMSEQ`. La fonction de choix lui attribue ensuite `SUMSEQ` ou `SUMPAR`. Dans le parcours `FORMEXEC`, les variables provenant du JSON sont substituées avant un nouveau calcul de la stratégie.

Une même formule paramétrique peut donc produire plusieurs programmes spécialisés : une petite instance séquentielle et une grande instance parallèle. Le compilateur structurel prend ensuite cette annotation comme donnée et génère le schéma correspondant.

Le développement Coq adopte une vue légèrement différente. Les constructeurs `ESumSeq` et `ESumPar` appartiennent déjà à la syntaxe source ; le modèle de coût C++ n'est pas formalisé. Les théorèmes répondent donc à la question conditionnelle « si cette stratégie est présente dans l'expression, sa compilation préserve-t-elle la valeur ? ». Ils ne prouvent pas que le choix est performant.

3.2 Modèle de coût SEQ/PAR

3.2.1 Estimation récursive du travail

Soit $\mathcal{C}(e)$ le coût estimé pour une expression e . L'implantation lui donne une définition syntaxique. Une constante ou une variable coûte une unité. Une opération ou un appel ajoute son coût propre aux coûts de ses arguments :

$$\mathcal{C}(c) = 1, \quad \mathcal{C}(x) = 1, \quad (3.1)$$

$$\mathcal{C}(f(e)) = c_f + \mathcal{C}(e), \quad \mathcal{C}(e_1 * e_2) = c_* + \mathcal{C}(e_1) + \mathcal{C}(e_2) \quad \text{où } (* = +, -, \text{ etc..}). \quad (3.2)$$

Un accès vectoriel coûte une unité en plus du coût du vecteur et de l'indice. Pour une somme imbriquée, $\mathcal{C}$ appelle récursivement l'estimation correspondant à sa stratégie déjà choisie. La mesure est donc une fonction de l'arbre et de la table de coûts.

Les bornes entières constituées de constantes et des opérateurs $+$, $-$ et $\times$ sont évaluées statiquement. Si elles valent a et b , le nombre d'itérations est :

$$N(a, b) = \max(b - a + 1, 0). \quad (3.3)$$

Si cette évaluation échoue, on emploie une taille arbitraire N_{def} . Cette approximation permet une décision dans le mode simple, mais elle peut naturellement être erronée pour la valeur d'exécution.

3.2.2 Coût séquentiel et coût parallèle simple

Pour une somme $\sum_{i=a}^b e$, on rappelle que l'estimation séquentielle est :

$$C_{\text{seq}} = \mathcal{C}(a) + \mathcal{C}(b) + N(a, b)(\mathcal{C}(e) + c_{\text{acc}}). \quad (3.4)$$

Les bornes sont calculées une fois, puis chaque itération évalue le corps et met à jour l'accumulateur.

Dans le cas parallèle simple on rappelle que :

$$C_{\text{par}} = \mathcal{C}(a) + \mathcal{C}(b) + \left\lceil \frac{N(a, b)}{\mathbf{P}} \right\rceil (\mathcal{C}(e) + c_{\text{acc}}) + \mathbf{P}\mathbf{g}V + \mathbf{L}. \quad (3.5)$$

Les notations suivent l'intuition BSP : $\mathbf{P}$ est le nombre de processus supposé, $\mathbf{g}$ le prix d'une unité communiquée, V le volume de réduction et $\mathbf{L}$ la synchronisation.

Pour une somme parallèle dont le corps contient déjà une somme parallèle, le code emploie une estimation particulière :

$$C_{\text{imb}} = \mathcal{C}(a) + \mathcal{C}(b) + \left\lceil \frac{N(a, b)}{G} \right\rceil (\mathcal{C}(e) + c_{\text{acc}}) + N(a, b) + G\mathbf{g}V + \mathbf{L}, \quad (3.6)$$

où G est un nombre supposé de groupes (voir [section 4.3.4](#)). Le terme N modélise l'affectation des itérations extérieures.

3.2.3 Exemple

Pour :

$$S(n) = \sum_{i=0}^n i, \quad (3.7)$$

Avec $n = 3$, $\mathcal{C}(i) = c_{\text{acc}} = 1$, $\mathbf{g} = 8$, $\mathbf{L} = 20$, $\mathbf{P} = 4$, $V = 1$, on obtient :

$$C_{\text{seq}} = 1 + 1 + 4(1 + 1) = 10, \quad (3.8)$$

$$C_{\text{par}} = 1 + 1 + \lceil 4/4 \rceil (1 + 1) + 32 + 20 = 56. \quad (3.9)$$

Donc $C_{\text{seq}} < C_{\text{par}}$

Avec $n = 99$:

$$C_{\text{seq}} = 1 + 1 + 100(1 + 1) = 202, \quad (3.10)$$

$$C_{\text{par}} = 1 + 1 + \lceil 100/4 \rceil (1 + 1) + 32 + 20 = 104. \quad (3.11)$$

Donc $C_{\text{par}} < C_{\text{seq}}$

3.3 Définition de la compilation

3.3.1 Principe général

Pour Γ, Φ, Ψ fixés, on abrège :

$$\mathbf{Comp}_n^{cp}(e) := \mathbf{Comp}(e, \Gamma, \Phi, \Psi, cp, n) = (c_e, r_e, \tau_e, n_e).$$

Le quadruplet donne le code c_e , la variable résultat r_e , son type τ_e et le prochain indice libre n_e ; les temporaires créés sont $\{\text{tmp}_k \mid n \leq k < n_e\}$. Le contexte $cp = (\text{rank}, \text{size}, \text{active}, \text{niveau}, \text{repr})$ décrit le groupe logique courant (au sens du groupe défini [section 4.3.4](#)) et la représentation des variables source. Dans $cp[i \mapsto x_i]$, la variable cible x_i représente l'indice lié i .

3.3.2 Règles structurelles

La compilation est définie par induction structurelle sur l'expression source : chaque cas compile récursivement ses sous-expressions, puis construit le code correspondant à l'expression complète.

Constante entière. Pour $z \in \mathbb{Z}$,

$$\mathbf{Comp}_n^{cp}(z) = (\text{tmp}_n := z, \text{tmp}_n, \text{Int}, n + 1).$$

Constante flottante. Pour $q \in \mathbb{R}$,

$$\mathbf{Comp}_n^{cp}(q) = (\text{tmp}_n := q, \text{tmp}_n, \text{Float}, n + 1).$$

Variables. Pour une variable x ,

$$\Gamma(x) = \text{Some}(\tau) \implies \mathbf{Comp}_n^{cp}(x) = (\mathbf{skip}, \text{repr}_{cp}(x), \tau, n).$$

Fonctions unaires. Le sous-terme est d'abord compilé, puis l'appel à $f^\sharp$ est placé dans un nouveau temporaire :

$$\frac{\mathbf{Comp}_n^{cp}(e) = (c, r, \tau_1, n_1) \quad \Phi(f, \tau_1) = \text{Some}(\tau)}{\mathbf{Comp}_n^{cp}(f(e)) = (c; \text{tmp}_{n_1} := f^\sharp(r), \text{tmp}_{n_1}, \tau, n_1 + 1)}.$$

Opérations arithmétiques binaires. Les appels récursifs consomment les noms frais de gauche à droite. Pour $t = \text{tmp}_{n_2}$, supposons

$$\mathbf{Comp}_n^{cp}(e_1) = (c_1, r_1, \tau_1, n_1) \quad \mathbf{Comp}_{n_1}^{cp}(e_2) = (c_2, r_2, \tau_2, n_2).$$

Pour $\odot \in \{+, \times\}$,

$$\mathbf{Comp}_n^{cp}(e_1 \odot e_2) = (c_1; c_2; t := r_1 \odot r_2, t, P(\tau_1, \tau_2), n_2 + 1).$$

Fonctions binaires.

$$\Psi(y, \tau_1, \tau_2) = \text{Some}(\tau) \implies \mathbf{Comp}_n^{cp}(y(e_1, e_2)) = (c_1; c_2; t := y^\sharp(r_1, r_2), t, \tau, n_2 + 1).$$

3.3.3 Compilation d'une somme séquentielle

Compilation des bornes. Les bornes sont compilées de gauche à droite :

$$\mathbf{Comp}_n^{cp}(a) = (c_a, r_a, \text{Int}, n_1), \quad \mathbf{Comp}_{n_1}^{cp}(b) = (c_b, r_b, \text{Int}, n_2).$$

Variables de la boucle. Les deux noms frais suivants désignent l'accumulateur et l'indice :

$$x_{\text{acc}} = \text{tmp}_{n_2}, \quad x_i = \text{tmp}_{n_2+1}.$$

Compilation du corps. Le corps est compilé dans les environnements étendus par l'indice i , représenté dans le code cible par x_i :

$$\mathbf{Comp}(e, \Gamma[i \mapsto \text{Int}], \Phi, \Psi, cp[i \mapsto x_i], n_2 + 2) = (c_e, r_e, \tau, n_3).$$

Ses temporaires sont :

$$T_e = \{\text{tmp}_k \mid n_2 + 2 \leq k < n_3\}.$$

La règle de compilation est :

$$\begin{aligned} C_{\text{seq}} := & \quad c_a; c_b; x_{\text{acc}} := 0_\tau; \\ & \mathbf{for}(x_i := r_a; \ x_i \leq r_b; \ x_i := x_i + 1) \{c_e; x_{\text{acc}} := x_{\text{acc}} + r_e\}^{[T_e]}, \\ \mathbf{Comp}_n^{cp} \left(\sum_{\substack{i=a \\ \text{seq}}}^b e \right) = & (C_{\text{seq}}, x_{\text{acc}}, \tau, n_3). \end{aligned}$$

L'annotation $[T_e]$ efface après chaque itération les temporaires du corps.

3.3.4 Compilation d'une somme parallèle simple

Lorsque le corps ne contient pas de SUMPAR, on conserve les prémisses précédentes et l'on pose :

$$x_{\text{loc}} = \text{tmp}_{n_2}, \quad x_i = \text{tmp}_{n_2+1}, \quad x_{\text{glob}} = \text{tmp}_{n_3}.$$

On obtient :

$$\begin{aligned} C_{\text{par}} := & \quad c_a; c_b; x_{\text{loc}} := 0_\tau; \\ & \mathbf{if} \ \text{active} \ \mathbf{then} \\ & \quad \mathbf{for}(x_i := r_a + \text{rank}; \ x_i \leq r_b; \ x_i := x_i + \text{size}) \\ & \quad \quad \{c_e; x_{\text{loc}} := x_{\text{loc}} + r_e\}^{[T_e]} \\ & \quad \mathbf{else skip}; x_{\text{glob}} := 0_\tau; \text{ALLREDUCE}(x_{\text{loc}}, x_{\text{glob}}), \\ \mathbf{Comp}_n^{cp} \left(\sum_{\substack{i=a \\ \text{par}}}^b e \right) = & (C_{\text{par}}, x_{\text{glob}}, \tau, n_3 + 1). \end{aligned}$$

Le rang r traite ainsi $a + r, a + r + p, \dots$, puis la réduction additionne les p accumulateurs locaux.

3.4 Énoncé de correction

On note $\iota(v)$ la case cible contenant la traduction de la valeur source v , par exemple $\iota(\text{VInt}(k)) = \text{Some}(\text{CVInt}(k))$. On pose $\perp = \text{None}$ et l'on note $\mathcal{E}_0$ l'ensemble des expressions sans SUMPAR. Pour $\Sigma = (\sigma_0, \dots, \sigma_{p-1})$, posons :

$$\begin{aligned} \Sigma \simeq_{cp}^n \rho &\iff \forall r < p, \quad \begin{cases} \forall y, v, \quad \rho(y) = \text{Some}(v) \implies \sigma_r(\text{repr}_{cp}(y)) = \iota(v), \\ \forall q \geq n, \quad \sigma_r(\text{tmp}_q) = \perp; \end{cases} \\ \langle \Sigma, c \cdot \kappa \rangle &:= \langle \sigma_0, c \cdot \kappa; \dots; \sigma_{p-1}, c \cdot \kappa \rangle. \end{aligned}$$

Théorème 3.1 (Correction du fragment sans somme parallèle). *Soient $e \in \mathcal{E}_0$, $e : \tau$ et $\rho \vdash e \Downarrow_\tau v$. Supposons que les primitives aient la même interprétation à la source et à la cible. Si*

$$\Sigma \simeq_{cp}^n \rho, \quad \mathbf{Comp}_n^{cp}(e) = (c, x, \tau, n'),$$

alors, pour toute continuation κ , il existe $\Sigma' = (\sigma'_0, \dots, \sigma'_{p-1})$ telle que :

$$\langle \Sigma, c \cdot \kappa \rangle \implies^* \langle \Sigma', \kappa \rangle, \quad \forall r < p, \quad \sigma'_r(x) = \iota(v).$$

Théorème 3.2 (Correction d'une somme parallèle simple). *Soit*

$$E = \sum_{\substack{i=a \\ \text{par}}}^b e : \tau, \quad a, b, e \in \mathcal{E}_0, \quad \rho \vdash E \Downarrow_\tau v.$$

Soient $p \geq 1$ processeurs et $\Sigma \simeq_{cp_0}^{n_0} \rho$, avec, pour tout $r < p$,

$$\sigma_r(\text{rank}) = \iota(\text{VInt}(r)), \quad \sigma_r(\text{size}) = \iota(\text{VInt}(p)), \quad \sigma_r(\text{active}) = \text{Some}(\text{CVBool}(\mathbf{true})).$$

Si $\oplus$ est associative et commutative, de neutre 0_τ , si les primitives coïncident et si

$$\mathbf{Comp}_{n_0}^{cp_0}(E) = (c, x, \tau, n'),$$

alors, pour toute continuation κ ,

$$\exists \Sigma', \quad \langle \Sigma, c \cdot \kappa \rangle \implies^* \langle \Sigma', \kappa \rangle \quad \wedge \quad \forall r < p, \quad \sigma'_r(x) = \iota(v).$$

Le périmètre démontré est donc :

$$\mathcal{E}_0 \cup \left\{ \sum_{\substack{i=a \\ \text{par}}}^b e \mid a, b, e \in \mathcal{E}_0 \right\}.$$

La démonstration complète de ces résultats de correction est dans le fichier `C08_Correctness.v`.

3.5 Preuve de correction du cas directeur

Cette section présente l'idée de la preuve du cas directeur : chaque rang calcule sa sous-somme locale, puis la réduction globale reconstitue la somme complète.

Nous rappelons que cet exemple est :

$$E_N = \sum_{\substack{i=0 \\ \text{par}}}^N f(i), \quad F(j) = \overline{\mathcal{U}_s}(f, \text{VInt}(j)), \quad S_N = \bigoplus_{j=0}^N F(j).$$

Proposition 3.3 (Correction du cas directeur). *Soient $N, n_0 \in \mathbb{N}$, $p \geq 1$, $\Sigma = (\sigma_0, \dots, \sigma_{p-1})$ et $\Phi(f, \text{Int}) = \text{Some}(\tau)$, avec $F(j) : \tau$ pour $0 \leq j \leq N$. On suppose $\oplus$ associative et commutative, de neutre 0_τ , ainsi que*

$$\begin{aligned} & \forall \mu, z, j, \quad 0 \leq j \leq N \wedge \mu(z) = \iota(\text{VInt}(j)) \implies \llbracket f^\sharp(z) \rrbracket_\mu = \iota(F(j)), \\ & \forall r < p, \quad \begin{cases} \sigma_r(\text{rank}) = \iota(\text{VInt}(r)), & \sigma_r(\text{size}) = \iota(\text{VInt}(p)), \\ \sigma_r(\text{active}) = \text{Some}(\text{CVBool}(\mathbf{true})), & \forall q \geq n_0, \quad \sigma_r(\text{tmp}_q) = \perp. \end{cases} \end{aligned}$$

Alors

$$\mathbf{Comp}_{n_0}^{cp_0}(E_N) = (c, x, \tau, n') \implies \forall \kappa, \exists \Sigma', \quad \langle \Sigma, c \cdot \kappa \rangle \implies^* \langle \Sigma', \kappa \rangle \wedge \forall r < p, \quad \sigma'_r(x) = \iota(S_N).$$

Idée de la preuve. La preuve suit les fragments successifs du code engendré.

Compilation. Notons c_0, c_N, c_f les codes compilés de $0, N, f(i)$, et r_0, r_N, r_f leurs variables résultats. La règle de la somme parallèle donne :

$$\begin{aligned} c = & \quad c_0; c_N; x_{\text{loc}} := 0_\tau; \\ & \mathbf{if} \text{ active } \mathbf{then} \\ & \quad \mathbf{for}(x_i := r_0 + \text{rank}; \ x_i \leq r_N; \ x_i := x_i + \text{size}) \\ & \quad \{c_f; x_{\text{loc}} := x_{\text{loc}} + r_f\} \quad \mathbf{else skip}; \\ & \quad x := 0_\tau; \mathbf{ALLREDUCE}(x_{\text{loc}}, x). \end{aligned}$$

Évaluation des bornes et du corps. La correction de ces trois sous-expressions donne, pour tout $r < p$,

$$\begin{aligned} & \sigma_r(r_0) = \iota(\text{VInt}(0)), \quad \sigma_r(r_N) = \iota(\text{VInt}(N)), \\ & \sigma_r(x_i) = \iota(\text{VInt}(j)) \implies \exists \sigma'_r, \quad \langle \sigma_r, c_f \cdot \kappa \rangle \rightarrow_\ell^* \langle \sigma'_r, \kappa \rangle \wedge \sigma'_r(r_f) = \iota(F(j)). \end{aligned}$$

Boucles locales. Pour $I_r = \{r + kp \mid k \in \mathbb{N}, r + kp \leq N\}$ et $K_r = |I_r|$, notons $\sigma_r^{(k)}$ la mémoire après k itérations. L'invariant est :

$$\mathcal{I}_r(k) : \quad \sigma_r^{(k)}(x_i) = \iota(\text{VInt}(r + kp)) \quad \wedge \quad \sigma_r^{(k)}(x_{\text{loc}}) = \iota\left(\bigoplus_{0 \leq q < k} F(r + qp)\right), \quad 0 \leq k \leq K_r.$$

L'initialisation établit $\mathcal{I}_r(0)$; le corps, l'accumulation et l'incrément donnent $\mathcal{I}_r(k) \Rightarrow \mathcal{I}_r(k+1)$. Donc

$$\sigma_r^{(K_r)}(x_{\text{loc}}) = \iota(L_r), \quad L_r = \bigoplus_{j \in I_r} F(j).$$

Réduction globale. Notons $\Sigma_L = (\sigma_0^{(K_0)}, \dots, \sigma_{p-1}^{(K_{p-1})})$; la division euclidienne par p donne :

$$\{0, \dots, N\} = \biguplus_{r=0}^{p-1} I_r.$$

Lorsque tous les rangs atteignent la réduction,

$$\langle \Sigma_L, \text{ALLREDUCE}(x_{\text{loc}}, x) \cdot \kappa \rangle \Longrightarrow \langle \Sigma', \kappa \rangle,$$

et, pour tout $r < p$,

$$\sigma'_r(x) = \iota \left(\bigoplus_{r=0}^{p-1} L_r \right) = \iota \left(\bigoplus_{j=0}^N F(j) \right) = \iota(S_N). \quad \square$$

On rappelle que Coq interprète les entiers dans $\mathbb{Z}$ et les flottants dans $\mathbb{R}$, alors que le générateur produit des `int` et des `double`. La preuve ignore donc les débordements, les valeurs non finies, l'arrondi et l'ordre de réduction MPI; l'associativité exacte employée plus haut ne vaut pas en IEEE 754. Elle suppose aussi la compatibilité des primitives source et cible. Elle établit ainsi une égalité dans le modèle abstrait, non une borne d'erreur ni une reproductibilité bit à bit. Ces deux garanties exigeraient une sémantique IEEE et une analyse numérique adaptées.

Chapitre 4

Implantation et évaluation

4.1 Objectifs et méthode d’observation

Ce chapitre confronte l’algorithme présenté au [chapitre 3](#) à son implantation effective. L’enjeu n’est pas seulement de décrire une suite de fichiers C++, mais de déterminer quelles propriétés de la chaîne complète sont réellement obtenues : quelles écritures $\text{\LaTeX}$ sont reconnues, à quel moment une somme devient séquentielle ou parallèle, comment les valeurs d’un document sont injectées dans le programme, et sous quelle forme le résultat revient dans ce même document. Le parseur repose sur ANTLR et est décrit dans l’[annexe A.1](#). Il faut noter que les expressions de type $2x, (x+1)(x-2)$ celles sans symboles $*$ sont plus délicates à traiter et nécessitent que le lexer insère automatiquement ces $*$. Mais il y a aussi des conditions particulières comme par exemple ne pas en ajouter entre une fonction f et ses arguments entre parenthèse : $f(3,4)$ n’est pas $f*(3,4)$

4.2 Architecture de la chaîne

4.2.1 Composants logiciels

L’implantation est organisée autour de quatre ensembles de sources. Le [Tableau 4.1](#) résume leur responsabilité principale.

Table 4.1 : Composants de l’implantation et responsabilités.

Composant	Rôle	Artefact principal
EXPR $\text{\LaTeX}$	Analyse lexicale et syntaxique, construction d’un premier arbre proche de la notation mathématique.	ExprNode
Pont bridge	Injection de la multiplication implicite, conversion entre AST, lecture du JSON, spécialisation et orchestration FORMEXEC.	ExprPtr, DonneesSection
compilateur	Typage du sous-langage, production de fragments en forme trois-adresses et assemblage du programme MPI.	fichier C++ autonome
FORMEXEC	Extraction depuis le document, numérotation, protocole d’entrée-sortie et restitution des résultats avec PythonTeX.	fichiers .formSkel.in/out

4.2.2 Deux parcours, un même cœur de compilation

Le premier mode analyse la formule, convertit son AST, infère les variables et les fonctions, vérifie le type, puis écrit le programme. Les variables libres sont initialisées à zéro et marquées par un commentaire ; les fonctions qui ne sont pas des normes natives reçoivent un corps qui lève une exception. Ce mode est donc utile pour examiner le code produit, mais il demande une intervention manuelle dès que la formule dépend de données ou de fonctions externes.

Le mode FORMEXEC ajoute au contraire un environnement d'exécution. Les paramètres sont entré par l'utilisateur dans une section JSON, les scalaires alors connus sont substitués dans l'AST avant le choix final de stratégie.

La Figure 4.1 présente le pipeline effectif.

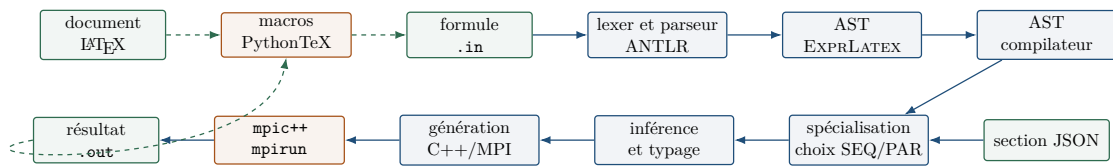


Figure 4.1 : Pipeline de l'implantation, de la formule au résultat réinséré dans le document.

Les macros PythonTeX extraient d'abord la formule du document et l'écrivent dans un fichier `.in`. ANTLR construit l'AST EXPR^ATeX, puis le pont le convertit dans l'AST du compilateur. La section JSON fournit les valeurs des variables, les fonctions C++, les nombres de processus les paramètres du modèle de coût, etc. Après spécialisation, le modèle de coût choisit SUMSEQ ou SUMPAR, le compilateur vérifie les types et génère le programme C++/MPI. Enfin, `mpic++` compile ce programme, `mpirun` l'exécute et le résultat écrit dans le fichier `.out` est relu par PythonTeX.

Par exemple, la configuration suivante peut être utilisée avec la formule $\sum_{i=0}^n \sin(xi)$ de la section 3 : les commentaires introduits par `//` indiquent le rôle des paramètres et doivent être retirés du fichier JSON réellement lu par le programme.

```

1 {
2   "sumStrategy": {
3     // Cout par défaut d'un appel de fonction unaire.
4     "unaryDefaultCost": 10.0,
5     // Cout par défaut d'un appel de fonction binaire.
6     "binaryDefaultCost": 15.0,
7     // Cout d'une operation arithmetique binaire.
8     "arithmeticBinaryCost": 1.0,
9     // Cout d'une addition dans l'accumulateur.
10    "accumulatorCost": 1.0,
11    // Cout g de communication d'une unite de donnee.
12    "communicationCost": 8.0,
13    // Latence L d'une synchronisation.
14    "synchronizationCost": 20.0,
15    // Nombre P de processus suppose par le modele.
16    "assumedProcesses": 4,
17    // Nombre minimal d'iterations pour autoriser SumPar.
18    "minParallelIterations": 8,
19    // Taille estimee si les bornes sont inconnues.
20    "unknownIntervalFallback": 32,
21    // Nombre de groupes pour les sommes imbriquees.
22    "nestedGroups": 2,
23    // Volume V de donnees traite par la reduction.

```

```

24   "reductionVolume": 1.0
25 },
26 "functions": [
27   {
28     "latex": "\\sin",
29     "c++": "std::sin",
30     "type": {
31       "input": ["R"],
32       "output": "R",
33       "cost": {"c++": 12.0}
34     }
35   }
36 ],
37 "environments": [
38   {
39     // On divise un fichier source latex en section
40     "section": 3,
41     "x": 0.5,
42     "n": [3, 20],
43     "processors": [1, 2, 4]
44   }
45 ]
46 }

```

Listing 4.1 : Exemple de configuration du pipeline.

4.3 Génération C++/MPI

4.3.1 Traduction par induction sur l'AST

La formule est d'abord transformée en AST. Le générateur suit ensuite la structure de cet arbre : pour compiler un nœud, il compile récursivement ses fils, puis construit l'instruction correspondant au nœud courant. À chaque expression e , il associe

$$\text{gen}(e) = (c_e, x_e, \tau_e),$$

où c_e est le code produit, x_e la variable contenant le résultat et τ_e son type. Les variables temporaires sont nommées `tmp0`, `tmp1`, etc.

Les cas de l'induction sont les suivants :

Constante (CstEnt, CstFlot).

Un entier n produit `int tmp = n` ; un réel produit une variable `double`.

Variable (Var).

Le résultat est directement le nom de la variable. Aucun code supplémentaire n'est émis.

Fonction unaire (FoncUnaire).

L'argument est compilé, puis le générateur émet `tmp = genFoncU_f(argument)`.

Par exemple, $\sin(x)$ appelle `genFoncU_sin(x)`.

Expression binaire (ExprBin).

Les opérandes sont compilés de gauche à droite, puis combinés. Les opérateurs `+`, `-` et `*` sont émis directement. Une fonction binaire $f(e_1, e_2)$ appelle `genFoncB_f`. La division convertit ses deux opérandes en `double` ; ainsi $7/2$ donne `3.5`.

Accès vectoriel (AccesVecteur).

Le vecteur et l'indice sont compilés, puis `genAccesVect(v,i)` effectue l'accès avec un contrôle de bornes.

Somme (Somme).

Les deux bornes sont compilées, l'indice est ajouté à l'environnement local, puis le corps est compilé dans la boucle. La forme de cette boucle dépend de la stratégie séquentielle ou parallèle.

Par exemple, pour :

$$3 + \sin(x) + \text{hypot}(v_i, 4.0),$$

le générateur produit le fragment suivant. Il contient une constante, deux variables, une fonction unaire, une fonction binaire, un accès vectoriel et des additions.

```
1 int tmp0 = 3;
2 double tmp1 = genFoncU_sin(x);
3 double tmp2 = (tmp0 + tmp1);
4 double tmp3 = genAccesVect(v, i);
5 double tmp4 = 4;
6 double tmp5 = genFoncB_hypot(tmp3, tmp4);
7 double tmp6 = (tmp2 + tmp5);
```

Listing 4.2 : Compilation des différents nœuds d'expression.

Les fonctions `genFoncU_sin` et `genFoncB_hypot` sont créées par le générateur. En mode FORMEXEC, leur corps est remplacé par la fonction C++ indiquée dans le fichier JSON, par exemple `std::sin` ou `std::hypot`.

4.3.2 Somme séquentielle

Une somme SUMSEQ évalue les bornes une seule fois, initialise un accumulateur avec le zéro de son type, puis exécute une boucle for. Le Listing 4.3 est le code produit pour $\sum_{i=0}^3 i$, sans les initialisations MPI qui entourent le calcul.

```
1 int tmp0 = 0;
2 int tmp1 = 3;
3 int tmp2 = 0;
4 for (int i = tmp0; i <= tmp1; ++i) {
5     tmp2 += i;
6 }
```

Listing 4.3 : Code généré pour la stratégie séquentielle.

Cette branche ne contient aucune communication. Si le programme est lancé avec plusieurs processus, chacun refait la même boucle et seul le rang zéro affiche le résultat.

4.3.3 Somme parallèle simple

Une somme SUMPAR est dite simple lorsque son corps ne contient aucune autre somme parallèle. Après l'évaluation unique de ses bornes a et b , elle distribue cycliquement l'intervalle. Dans un contexte $(r, \mathbf{P}, A, \ell)$, r est le rang local, $\mathbf{P}$ le nombre de processus disponibles, A leur condition d'activité et ℓ le niveau d'imbrication de la somme parallèle. Le rang r ne travaille que si A est vrai et traite :

$$a + r, a + r + \mathbf{P}, a + r + 2\mathbf{P}, \dots$$

tant que l'indice reste inférieur ou égal à b . Chaque processus accumule une valeur locale, puis `MPI_Allreduce` combine toutes les contributions avec `MPI_SUM` et diffuse le résultat à tous les rangs. Le Listing 4.4 reproduit le fragment effectivement généré pour :

$$S = \sum_{i=0}^{20} i = 210.$$

```

1  int tmp0 = 0;
2  int tmp1 = 20;
3  int tmp2 = 0;
4  if (true) {
5      for (int i = tmp0 + rang; i <= tmp1; i += nbProc) {
6          tmp2 += i;
7      }
8  }
9  int tmp3 = 0;
10 MPI_Allreduce(&tmp2, &tmp3, 1,
11              MPI_INT, MPI_SUM, MPI_COMM_WORLD);

```

Listing 4.4 : Code généré pour une somme parallèle simple.

Les progressions des rangs sont disjointes et couvrent tout l'intervalle. Le type de la réduction est `MPI_INT` ou `MPI_DOUBLE` selon le type du corps.

4.3.4 Sommes parallèles imbriquées

Considérons :

$$S = \sum_{i=a}^b \sum_{j=c(i)}^{d(i)} f(i, j). \quad (4.1)$$

La somme sur i est dite *extérieure* ; pour chaque valeur de i , la somme sur j est dite *intérieure*.

Le générateur détecte que le corps de la somme extérieure contient une somme parallèle. Il divise alors les processus en deux groupes logiques lorsque $\mathbf{P} > 1$, et en un seul groupe lorsque $\mathbf{P} = 1$. Si $G \in \{1, 2\}$ est ce nombre de groupes et $h = \lceil \mathbf{P}/G \rceil$, le rang global r reçoit :

$$\gamma = \min\left(\left\lfloor \frac{r}{h} \right\rfloor, G - 1\right), \quad \rho = r - \gamma h, \quad \mathbf{P}_\gamma = \min(h, \mathbf{P} - \gamma h).$$

Ici, γ est le numéro du groupe, ρ le rang dans ce groupe et $\mathbf{P}_\gamma$ sa taille.

Par exemple, avec $\mathbf{P} = 7$ processus, on obtient $G = 2$ et $h = \lceil 7/2 \rceil = 4$. Les processus sont répartis ainsi :

$$\underbrace{\{0, 1, 2, 3\}}_{\gamma=0, \mathbf{P}_0=4} \quad \underbrace{\{4, 5, 6\}}_{\gamma=1, \mathbf{P}_1=3}.$$

Dans le premier groupe, les rangs locaux ρ valent 0, 1, 2, 3. Dans le second, les processus globaux 4, 5, 6 reçoivent respectivement les rangs locaux 0, 1, 2.

Tous les processus parcourent les valeurs de i dans le même ordre. La valeur i est confiée au groupe :

$$\text{owner}(i) = ((i - a) \bmod G + G) \bmod G.$$

Le *groupe propriétaire* de i est le groupe de numéro $\text{owner}(i)$: il est le seul chargé de calculer la somme intérieure associée à cette valeur de l'indice extérieur. Le corps est ensuite compilé

récurivement avec le contexte :

$$(\rho, \mathbf{P}_\gamma, A \wedge \gamma = \text{owner}(i), \ell + 1).$$

La somme intérieure est donc répartie entre les rangs du groupe propriétaire ; les autres rangs accumulent zéro. Tous appellent néanmoins `MPI_Allreduce` sur `MPI_COMM_WORLD`, ce qui conserve le même ordre de collectives sur chaque processus.

Après la réduction intérieure, seul le rang $\rho = 0$ du groupe propriétaire ajoute le résultat à l'accumulateur extérieur. Une dernière réduction réunit les accumulateurs. Chaque indice extérieur appartient ainsi à un seul groupe, et chaque indice intérieur à un seul rang de ce groupe : chaque terme $f(i, j)$ est ajouté exactement une fois.

Exemple pour la formule :

$$S = \sum_{i=0}^{20} \sum_{j=0}^{20} (i + j), \quad S = 8820,$$

le modèle de coût choisit la stratégie parallèle pour les deux sommes. Le fragment suivant est celui produit dans le `main`.

```

1 // Bornes de la somme exterieure sur i.
2 int tmp0 = 0;
3 int tmp1 = 20;
4
5 // G = nombre de groupes; h = taille maximale d'un groupe.
6 int tmp3 = (nbProc > 1) ? 2 : 1;
7 int tmp4 = (nbProc + tmp3 - 1) / tmp3;
8
9 // gamma = groupe du processus courant.
10 int tmp5 = rang / tmp4;
11 if (tmp5 >= tmp3) {
12     tmp5 = tmp3 - 1;
13 }
14
15 // rho = rang local; p_gamma = taille reelle du groupe.
16 int tmp6 = rang - (tmp5 * tmp4);
17 int tmp7 = std::min(tmp4, nbProc - (tmp5 * tmp4));
18
19 // Accumulateur local de la somme exterieure.
20 int tmp2 = 0;
21 for (int i = tmp0; i <= tmp1; ++i) {
22     // Groupe propriétaire de la valeur courante de i.
23     int tmp8 = ((i - tmp0) % tmp3 + tmp3) % tmp3;
24
25     // Bornes et accumulateur local de la somme interieure.
26     int tmp10 = 0;
27     int tmp11 = 20;
28     int tmp12 = 0;
29
30     // Seul le groupe propriétaire repartit les valeurs de j.
31     if (true && tmp5 == tmp8) {
32         for (int j = tmp10 + tmp6;
33             j <= tmp11; j += tmp7) {
34             int tmp14 = (i + j);
35             tmp12 += tmp14;
36         }

```

```

37     }
38
39     // Reunion des parts de la somme interieure.
40     int tmp13 = 0;
41     MPI_Allreduce(&tmp12, &tmp13, 1,
42                 MPI_INT, MPI_SUM, MPI_COMM_WORLD);
43
44     // Un seul rang ajoute le resultat interieur.
45     if (true && tmp5 == tmp8 && tmp6 == 0) {
46         tmp2 += tmp13;
47     }
48 }
49
50 // Reunion finale des parts de la somme exterieure.
51 int tmp9 = 0;
52 MPI_Allreduce(&tmp2, &tmp9, 1,
53             MPI_INT, MPI_SUM, MPI_COMM_WORLD);

```

Listing 4.5 : Code généré pour deux sommes parallèles imbriquées.

Dans les deux conditions, `true` est la condition d'activité héritée du contexte extérieur. Elle vaut vrai ici puisque tous les processus sont actifs à la racine ; le générateur la conserve néanmoins afin d'utiliser le même schéma de code dans un contexte imbriqué où cette condition peut être restrictive.

Avec quatre processus, les rangs 0, 1 forment le premier groupe et les rangs 2, 3 le second. Les groupes traitent alternativement les valeurs de i . La première réduction calcule chaque somme intérieure ; la dernière calcule leur somme. Le programme renvoie 8820 avec un à cinq processus.

Le générateur utilise actuellement deux groupes logiques et toutes les réductions portent sur `MPI_COMM_WORLD`. Cette méthode est correcte sur les exemples testés, mais les sommes parallèles imbriquées ne sont pas couvertes par les preuves Coq.

4.3.5 Assemblage du programme

Le fragment final est placé dans un programme qui inclut les bibliothèques C++ et `mpi.h`. Le `main` initialise MPI, récupère `rang` et `nbProc`, déclare les variables libres, exécute le fragment, affiche le résultat au rang zéro, puis termine MPI. Les fonctions générées et `genAccesVect` sont placées avant le `main`.

Par exemple, pour la formule $3 + 2 \times 5$, le programme complet prend la forme simplifiée suivante :

```

1  #include <iostream>
2  #include <mpi.h>
3
4  int main(int argc, char** argv) {
5      MPI_Init(&argc, &argv);
6
7      int rang = 0;
8      int nbProc = 0;
9      MPI_Comm_rank(MPI_COMM_WORLD, &rang);
10     MPI_Comm_size(MPI_COMM_WORLD, &nbProc);
11
12     int tmp0 = 3;
13     int tmp1 = 2;
14     int tmp2 = 5;

```

```
15     int tmp3 = (tmp1 * tmp2);
16     int tmp4 = (tmp0 + tmp3);
17
18     if (rang == 0) {
19         std::cout << tmp4 << std::endl;
20     }
21
22     MPI_Finalize();
23     return 0;
24 }
```

Listing 4.6 : Assemblage C++/MPI pour une expression simple.

Chaque processus calcule ici `tmp4`, mais seul le rang zéro affiche la valeur 13.

Chapitre 5

Conclusion

5.1 Bilan

Ce mémoire partait d’une difficulté concrète de la programmation scientifique : comment proposer un outil qui simplifie l’écriture, favorise la correction du code et permette de reproduire un calcul en prenant les mathématiques $\text{\LaTeX}$ comme point d’entrée ?

Le résultat apporte une réponse positive, mais circonscrite. Pour le fragment effectivement implanté, une formule peut jouer trois rôles à la fois : notation lisible dans l’article, spécification d’un calcul et source d’un programme C++/MPI. La chaîne FORMEXEC extrait la formule, associe les données de la configuration, spécialise l’expression, choisit une stratégie, produit un programme, le compile, l’exécute et rend son résultat accessible au document. Le cas directeur

$$\sum_{i=0}^n f(i)$$

devient une représentation suffisamment structurée pour engendrer soit une boucle locale, soit une partition cyclique suivie d’un ALLREDUCE. La variation de n peut changer ce choix sans demander à l’auteur de réécrire la formule, comme l’illustre le chapitre d’implantation. Cette génération est accompagnée d’une certification en Coq : pour le fragment séquentiel et une somme parallèle simple à la racine, les théorèmes de correction établissent que le code cible calcule la même valeur que la sémantique source, sous les hypothèses précisées dans le chapitre de compilation.

5.2 limitation et future travaux

Cette réponse ne signifie toutefois pas qu’un utilisateur puisse déjà exprimer tout calcul scientifique avec « seulement des mathématiques ». Les valeurs, les fonctions disponibles, les nombres de processus et les paramètres du coût sont encore décrits dans un fichier JSON ; Le parser accepte encore actuellement plus d’expression que le compilateur ne peut traiter. ; et les vecteurs restent répliqués. De plus, l’addition des nombres flottants selon la norme IEEE 754 n’est pas associative à cause des arrondis : une réduction parallèle, qui modifie l’ordre des additions, peut donc produire un résultat légèrement différent de celui du calcul séquentiel.

Les matrices, tenseurs, conditions dans les sommes, communications de tableaux, GPU et vectorisation SIMD sont hors du périmètre actuel. La formalisation Coq est plus étroite encore : elle ne contient pas les vecteurs et sa correction parallèle concerne une somme parallèle simple à la racine. Cette dissociation entre prototype et modèle formel a été maintenue tout au long du

mémoire ; elle empêche d'attribuer à l'implantation une garantie que la preuve n'établit pas.

Références

- [1] J. DEMMEL et H. D. NGUYEN. “Fast Reproducible Floating-Point Summation”. In : *2013 IEEE 21st Symposium on Computer Arithmetic*. IEEE, 2013, p. 163-172. DOI : [10.1109/ARITH.2013.9](https://doi.org/10.1109/ARITH.2013.9).
- [2] D. GOLDBERG. “What Every Computer Scientist Should Know About Floating-Point Arithmetic”. In : *ACM Computing Surveys* 23.1 (1991), p. 5-48. DOI : [10.1145/103162.103163](https://doi.org/10.1145/103162.103163).
- [3] D. E. KNUTH. “Literate Programming”. In : *The Computer Journal* 27.2 (1984), p. 97-111. DOI : [10.1093/comjnl/27.2.97](https://doi.org/10.1093/comjnl/27.2.97).
- [4] MESSAGE PASSING INTERFACE FORUM. *MPI : A Message-Passing Interface Standard, Version 5.0*. Juin 2025. URL : <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf> (visité le 30/08/2026).
- [5] T. J. PARR, S. HARWELL et K. FISHER. “Adaptive LL(*) Parsing : The Power of Dynamic Analysis”. In : *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*. ACM, 2014, p. 579-598. DOI : [10.1145/2660193.2660202](https://doi.org/10.1145/2660193.2660202).
- [6] THE COQ DEVELOPMENT TEAM. *The Coq Proof Assistant, Reference Manual, Version 8.20*. 2024. DOI : [10.5281/zenodo.14542673](https://doi.org/10.5281/zenodo.14542673). URL : <https://rocq-prover.org/doc/V8.20.1/refman/> (visité le 30/08/2026).
- [7] L. G. VALIANT. “A Bridging Model for Parallel Computation”. In : *Communications of the ACM* 33.8 (1990), p. 103-111. DOI : [10.1145/79173.79181](https://doi.org/10.1145/79173.79181).

Annexe A

Compléments techniques

La conception et la mise en œuvre du parseur ANTLR ont été réalisées par mon tuteur de stage.

A.1 Front-end ANTLR

A.1.1 Choix d'un analyseur généré

Le front-end repose sur deux grammaires ANTLR séparées, une grammaire lexicale et une grammaire syntaxique. ANTLR fournit ici le mécanisme de reconnaissance et le code C++ du lexer et du parseur ; les actions sémantiques embarquées dans la grammaire construisent directement le premier AST [5]. Ce choix évite d'écrire manuellement un analyseur récursif tout en conservant dans le dépôt une spécification lisible des précédences.

Le lexer ignore les espaces, retours à la ligne, commentaires introduits par % et quelques commandes d'espacement usuelles. Il distingue les entiers, les nombres décimaux, une forme exponentielle, les opérateurs directs et plusieurs commandes L^AT_EX. Le Listing A.1 reproduit un extrait exact de la grammaire.

```
1  tokens { IMUL }
2
3  WS : [ \t\r\n]+ -> skip;
4  LINE_COMMENT: '%' ~[\r\n]* -> skip;
5
6  BigSUM : BACKSLASH 'sum';
7  INT    : DIGIT+ ;
8  DOUBLE : DIGIT+ '.' DIGIT* | '.' DIGIT+ ;
9
10 MUL      : '*' | BACKSLASH 'times';
11 ADD      : '+';
12 MINUS    : '-';
13 CIRCUMFLEX : '^';
14 SLASH    : '/';
15
16 IDENT    : LETTER;
17 FUNCTION : BACKSLASH LETTER+;
18 LETTER   : [a-zA-Z];
```

Listing A.1 : Extrait du lexer de formules.

La règle IDENT montre une frontière structurelle : une variable est une lettre latine unique. Une commande comme \alpha devient un jeton de fonction, non un identifiant. Le tableau

`letters` du JSON énumère bien des lettres grecques, mais aucune lecture de ce tableau n'est effectuée lors de l'analyse. Étendre les identifiants demande donc une modification du lexer ou un prédicat lexical alimenté par la configuration.

A.1.2 Insertion de la multiplication implicite

La notation mathématique omet fréquemment le symbole de multiplication : $2x$, $2(x + 1)$ ou $(x - y)(z + 1)$. Le lexer ne peut pas décider isolément si deux jetons consécutifs doivent être séparés par une multiplication. Le pont interpose donc une source de jetons qui parcourt le flux brut et ajoute un jeton synthétique `IMUL` entre certaines catégories.

Neuf catégories sont considérées : nombre, variable, parenthèse ouvrante, parenthèse fermante, grande somme, accolade ouvrante, accolade fermante, factorielle et fonction. Une matrice booléenne encode les couples admissibles. Par exemple, nombre–variable et variable–parenthèse produisent une multiplication ; fonction–parenthèse n'en produit pas.

Cette passe est volontairement locale : elle ne reconstruit pas le contexte $\text{\LaTeX}$ complet. Le code source signale deux ambiguïtés qui en découlent. Une accolade fermante peut terminer la borne d'une somme ou un groupement destiné à être multiplié. De même, $f(x)$ peut désigner un appel ou le produit d'une variable f par une parenthèse. Dans le corpus actuel, les fonctions configurées sont écrites avec une commande telle que `\sin` ou `\hypot`, ce qui lève cette seconde ambiguïté. Pour un langage plus large, la décision devrait s'appuyer sur une table des symboles ou sur un prétraitement structuré.

A.2 Intégration documentaire avec FORMEXEC

A.2.1 La formule comme source et comme élément du document

Le paquet `formulaToSkel.sty` met en œuvre une forme de programmation lettrée : le texte scientifique contient la formule source, la demande d'exécution et les commandes qui relisent son résultat. PythonTeX fournit le mécanisme qui exécute du Python au cours de la construction du document et restitue du contenu $\text{\LaTeX}$.

La macro principale reçoit la formule comme second argument. Dans le mode C++, elle fixe les options, transmet le texte à Python, enregistre le label et la langue, puis prépare le traitement de fin de document. Le Listing A.2 en présente le noyau exact.

```

1 \newcommand{\formulaToSkel}[2] [] {%
2   \setkeys{formToSkel}{
3     visible = true,
4     language = Cplusplus,
5     label =,
6     #1
7   }
8   \pyc{formToSkel.formulaToSkel(r'#2')}
9   \begingroup\edef\x{\endgroup
10    \noexpand\pyc{formToSkel.formulaToSkelBis(
11      r'\cmdKV@formToSkel@label',
12      r'\cmdKV@formToSkel@language',r'\showlineno')}}\x
13 }
```

Listing A.2 : Noyau de la macro `formulaToSkel`.

Le script Python retire les dollars extérieurs et les commentaires non échappés, incrémente les compteurs de section et de formule, puis écrit un fichier nommé `sectionX.formulaY.formSkel`.

in. Si le contenu diffère d’une version existante, l’ancienne sortie est supprimée afin d’éviter qu’un résultat obsolète soit affiché. Une table séparée associe les labels aux couples section–formule ; les macros de restitution peuvent ainsi viser un label plutôt qu’un numéro fragile.

A.2.2 Cycle de construction

La chaîne documentaire nécessite plusieurs étapes, car le document doit d’abord matérialiser les entrées, puis attendre le compilateur, puis relire les sorties. La séquence recommandée est :

```

1 cmake -S . -B build
2 cmake --build build --target formexec_bridge_driver
3
4 cd FormExec
5 pdflatex -shell-escape article.tex
6 cd ..
7
8 ./build/formexec_bridge_driver \
9   FormExec/formSkel FormExec/config.json build/formexec_runs
10
11 cd FormExec
12 pdflatex -shell-escape article.tex
13 pdflatex -shell-escape article.tex

```

Listing A.3 : Construction de bout en bout du démonstrateur.

Le paquet exige `-shell-escape`. À la fin du document, il invoque `pythontex` avec Python 3, puis exécute un script qui finalise les compteurs et fichiers d’environnement. Cette automatisation améliore la traçabilité, mais le droit d’exécuter des commandes arbitraires implique que le document et ses scripts doivent être considérés comme du code.

A.2.3 Complétion, compilation et protocole de sortie

Pour un cas donné, le pont produit d’abord un programme générique. Il parcourt ensuite son texte pour remplacer les corps marqueurs des fonctions configurées et les déclarations marquées “variable libre à modifier”. En pratique, les scalaires ont déjà été substitués dans l’AST ; cette seconde opération sert principalement aux vecteurs.

Le programme est compilé par une commande `mpic++ source -o executable`. Le pilote lance ensuite successivement `mpirun -np p executable` pour chaque élément de la liste `processors`. La sortie standard, à laquelle la sortie d’erreur est jointe, devient la valeur du cas. Un échec de génération, de compilation ou d’exécution produit un fichier commençant par **ERROR**.

Deux formats normaux sont utilisés :

1 SOLO	MULTI
2 1,2,4	1 2 4
3 valeur_p1	valeur_cas1_p1
4 temps_ns_p1	temps_cas1_p1
5 valeur_p2	valeur_cas2_p1
6 temps_ns_p2	temps_cas2_p1
7 ...	valeur_cas1_p2
8	temps_cas1_p2
9	...

Listing A.4 : Protocoles de sortie consommés par FORMEXEC.

Dans un fichier MULTI, les blocs sont ordonnés d'abord par nombre de processus, puis par cas. Les macros `\formulaToSkelValue` et `\formulaToSkelTime` lisent le format SOLO. Les variantes `\formulaToSkelNthValue` et `\formulaToSkelNthTime` sélectionnent le cas et la position du nombre de processus dans le format MULTI.

A.3 Support initial des vecteurs

A.3.1 Représentation et accès

Le JSON distingue un vecteur d'une variable MULTI en l'entourant d'un objet :

```

1  "v": {"values": [1.0, 2.5, 3.0, 4.0]}
2
3  "w": [
4    {"values": [1, 2, 3]},
5    {"values": [1, 2, 3, 4, 5]}
6  ]

```

Listing A.5 : Vecteur fixe et vecteurs MULTI.

Le lecteur convertit chaque composante avec la même règle numérique que les scalaires. Si toutes sont entières, le vecteur devient `std::vector<int>`; sinon il devient `std::vector<double>`.

Un accès appelle la fonction générique `genAccesVect`. Celle-ci vérifie que l'indice est positif ou nul et strictement inférieur à la taille, puis retourne une référence constante vers l'élément. Le contrôle est donc effectué à l'exécution sur chaque accès. L'indice est en base zéro dans les deux syntaxes `v[i]` et `v_i`.

La normes euclidienne est disponibles. Pour un vecteur, elles ont actuellement le corps :

$$\sqrt{\sum_{k=0}^{m-1} v_k^2}.$$

Pour un scalaire, elle devient la valeur absolue.

A.3.2 Chargement texte et Matrix Market

Un vecteur peut référencer un fichier :

```

1  "u": {
2    "file": "data/external-vector.mtx",
3    "reading": "partial"
4  }

```

Listing A.6 : Descripteur d'un vecteur externe.

Les chemins relatifs sont résolus par rapport au dossier de `config.json`. Le lecteur accepte d'abord un format texte simple : les nombres peuvent être séparés par des espaces, retours à la ligne ou virgules, et les commentaires commencent par `#` ou `%`.

Si la première ligne commence par `%%MatrixMarket`, le pont interprète un format `array` ou `coordinate`, à champ `integer` ou `real`. Il accepte les objets annoncés comme vecteur ou covecteur, ainsi qu'une matrice dont l'une des dimensions vaut un. Dans le format coordonné, les indices en base un sont convertis en base zéro et les composantes absentes sont initialisées à zéro. Cette compatibilité s'appuie sur le format d'échange Matrix Market.

Le mot **partial** ne correspond cependant pas encore à une lecture partielle.

A.3.3 Périmètre exact

Le support courant permet :

- des vecteurs d’entiers ou de flottants ;
- une valeur fixe ou une séquence MULTI de vecteurs ;
- un chargement texte ou Matrix Market ;
- l’accès par indice, y compris un indice calculé ou imbriqué ;
- une somme scalaire des éléments ;
- les normes euclidienne.

Il ne permet pas encore les opérations sur un vecteur complet, une somme dont le résultat serait vectoriel, une réduction MPI de plusieurs composantes, une distribution des données, les matrices ni les tenseurs. Les rubriques “Vector/Matrices” de la grammaire restent vides.

A.4 Exemples implémentés

Le document `FormExec/article.tex` contient 39 formules réparties en six sections. Le Tableau A.1 détaille les familles couvertes et le mode de données.

Table A.1 : Composition du corpus de démonstration.

Section	Famille	Formules	Sortie	Processus demandés
1	constantes, priorités, multiplication implicite, division, puissance, fonctions et norme scalaire	13	SOLO	1
2	variables scalaires fixes	6	SOLO	1
3	sommes simples, trigonométriques et imbriquées, avec $n \in \{3, 20\}$	6	MULTI	1, 2, 4
4	accès et normes de vecteurs fixes	8	SOLO	1, 2, 4
5	deux vecteurs et bornes MULTI	4	MULTI	1, 2, 4
6	vecteur externe Matrix Market	2	SOLO	1, 2, 4
Total		39	29 SOLO, 10 MULTI	

Un parcours propre construit logiquement 49 programmes : un pour chacune des 29 formules SOLO et deux pour chacune des 10 formules MULTI. Les listes de processus conduisent à 109 lancements MPI.

Au moment de l’audit, les 39 entrées possédaient chacune un fichier de sortie ; 29 commençaient par **SOLO**, 10 par **MULTI**, et aucune par **ERROR**. Les valeurs calculées pour les cas directeurs correspondent aux valeurs annoncées dans le document.